



**UNIVERSIDAD
NACIONAL DE
INGENIERÍA**



TRANSFORMACIÓN
digital

CURSOS GRATUITOS

OTI UNI

Ansible desde Cero: Automatiza Linux y Servidores

Cursos PIT – 9na Edición

Instructor:  **Anthony Gonzalo Quispe Cordova**

Oficina de Tecnologías de la Información (OTI)

30 de mayo de 2026



Flujo de Cursos Orientado a DevOps



Horarios de los Cursos del Flujo DevOps



Fundamentos de Linux



Sábados



9:00 a.m. – 12:00 p.m.



23/05 – 27/06 | 18 h



Git y GitHub



Lun, Mar, Jue, Vie



4:00 – 7:00 p.m.



19/05 – 28/05 | 18 h



Fundamentos de Docker



Lun, Mar, Vie



4:00 – 7:00 p.m.



29/05 – 09/06 | 18 h



Ansible: Automatización



Sábados



2:00 – 5:00 p.m.



23/05 – 27/06 | 18 h

Índice de Contenidos

- 1 Bloque 1: ¿Qué es Ansible?
- 2 Bloque 2: Introducción a YAML
- 3 Bloque 3: Preparación del Laboratorio
- 4 Bloque 4: Comandos Ad-Hoc
- 5 Bloque 5: Primer Playbook

Bloque 1: ¿Qué es Ansible?

El Problema: Administración Manual de Servidores

- Instalar paquetes **uno por uno** en cada servidor.
- Configurar servicios **manualmente** y rezar para no olvidar un paso.
- Repetir **exactamente** lo mismo en 10, 50 o 100 máquinas.
- Un error humano → horas de depuración.

Resultado

Procesos lentos, inconsistentes y propensos a errores.



¿Te suena familiar?

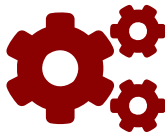
La Solución: Ansible

¿Qué es Ansible?

Motor de **automatización de TI** de código abierto, creado por Michael DeHaan (2012) y mantenido por **Red Hat**.

¿Para qué sirve?

- **Gestión de configuración** de servidores.
- **Despliegue** de aplicaciones de forma repetible.
- **Orquestación** de tareas en múltiples máquinas.



"Documentación ejecutable."

Ansible en Contexto

Origen

Ansible fue creado en **2012** por Michael DeHaan con una idea central: automatizar servidores de forma simple, sin instalar agentes remotos.

- En **2015**, Red Hat adquiere Ansible.
- Evoluciona de Ansible Tower a **Ansible Automation Platform**.
- Hoy automatiza Linux, Windows, redes, cloud y seguridad.



Simplicidad

+ SSH + YAML + módulos

Scripts vs Ansible

Scripts tradicionales	Ansible
Ejecutan pasos de forma secuencial.	Declara el estado deseado .
Requieren controlar muchos casos manualmente.	Usa módulos preparados para tareas comunes.
Pueden ser difíciles de reutilizar en equipos grandes.	Playbooks legibles y versionables.
El resultado depende mucho de cómo se escribió el script.	Busca ser idempotente : repetir no debería romper.

Ansible no reemplaza todos los scripts; los organiza dentro de una automatización más clara, auditable y repetible.

¿Por qué elegir Ansible?

Agentless

No requiere instalar nada
en los servidores remotos

YAML legible

Playbooks en lenguaje
simple y entendible

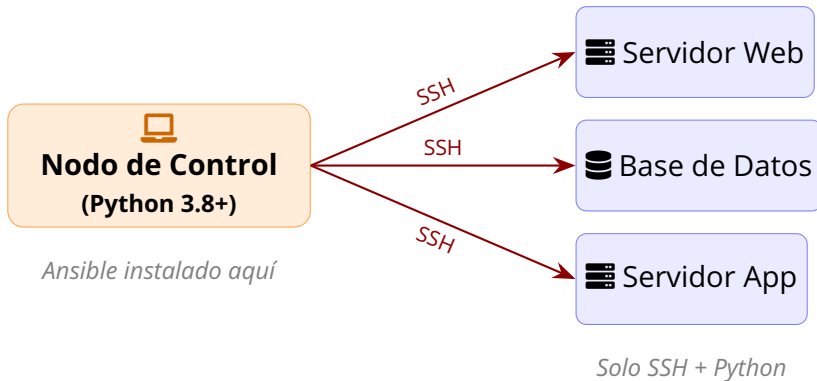
Solo necesita SSH

Conexión segura estándar
ya disponible en Linux

Idempotente

Solo aplica cambios si el
sistema no está en
el estado deseado

Arquitectura Agentless de Ansible



Modelo Push: Ansible empuja configuraciones vía SSH. Sin agentes ni demonios en los servidores = menor superficie de ataque.

Bloque 2: Introducción a YAML

Formatos de Datos: XML, JSON y YAML

XML	JSON	YAML
Muy formal, basado en etiquetas.	Ligero, común en APIs y aplicaciones web.	Legible para humanos, usado en automatización.
Verboso para archivos de configuración.	Usa llaves, corchetes y comillas con frecuencia.	Usa indentación, listas y pares clave-valor.

¿Por qué YAML en Ansible?

Los playbooks deben ser fáciles de leer, revisar y versionar. YAML permite expresar tareas, variables y estructuras sin demasiado ruido visual.

Reglas Básicas de YAML

Ideas clave

- Los comentarios empiezan con #.
- La estructura depende de la **indentación**.
- Se recomiendan **2 espacios**; no usar tabs.
- Las listas usan guion: - item.
- Las variables suelen ser clave: valor.

Ejemplo corto

```
# Datos del servidor
servidor: web01
puerto: 80
activo: true
paquetes:
  - nginx
  - curl
```

Secuencias, Mapas y Escalares

Escalares

Valores simples: texto, números, booleanos o nulos.

Mapas

Pares clave: valor, parecidos a un diccionario.

Secuencias

Listas ordenadas de elementos.

Mezcla común

```
usuario:
  nombre: ansible
  sudo: true
  grupos:
    - admins
    - devops
contacto: {ciudad: Lima, pais: Peru}
```

Estructura Común de un Playbook

Playbook mínimo

```
---  
- name: Preparar servidor web  
  hosts: web  
  become: true  
  tasks:  
    - name: Instalar nginx  
      apt:  
        name: nginx  
        state: present
```

- `hosts`: indica el grupo del inventario.
- `become`: permite ejecutar con privilegios

Mini Actividad: Autobiografía en YAML

Crea un archivo `autobiografia.yaml` para practicar estructura e indentación.

Plantilla sugerida

```
---
# Autobiografia en YAML
nombre: "Tu nombre"
contacto: {correo: "tu@correo.com", ciudad: "Lima"}
habilidades:
  - Linux
  - Git
  - Ansible
metas:
  corto_plazo: "Practicar automatizacion"
  largo_plazo: "Administrar servidores"
# Fin del documento
```

Bloque 3: Preparación del Laboratorio

Herramientas Necesarias para el Laboratorio

VirtualBox y Ubuntu

Usaremos una **única máquina virtual** como entorno aislado principal para nuestro laboratorio.

Docker + Dive Into Ansible

Dentro de la VM, Docker levantará los múltiples nodos (*contenedores*) que gestionaremos con Ansible.



VM Ubuntu (VirtualBox)



Contenedores (Nodos)

Instalación de Ansible por Sistema Operativo

Ubuntu / Debian

```
sudo apt update  
sudo apt install ansible -y
```

RHEL / CentOS / Fedora

```
sudo dnf install ansible -y
```

macOS

```
brew install ansible
```

En nuestro laboratorio

El contenedor ubuntu-c ya trae Ansible instalado; solo verificaremos con `ansible -version`.

Arquitectura del Laboratorio Multi-Servidor



Objetivo

Simular un entorno real con múltiples servidores para practicar Ansible sin riesgo.

Preparación del Entorno Base

1. Instalar VirtualBox

- Descargar e instalar VirtualBox en tu computadora anfitriona.

2. Crear la VM Ubuntu Base

- Descargar la ISO de **Ubuntu Server 22.04** o superior.
- Crear una VM con al menos **2 GB de RAM** y **2 CPUs**.
- Configurar el adaptador de red en modo **Adaptador Sólo Anfitrión o Red Interna** para tener conexión con la VM.
- Instalar el sistema operativo.

Levantando el Laboratorio en la VM

Pasos dentro de la VM Ubuntu:

- 1 Instalar Docker y Git: `sudo apt update && sudo apt install docker.io docker-compose-v2 git -y`
- 2 Clonar el repositorio: `git clone https://github.com/spurin/diveintoansible-lab.git`
- 3 Iniciar el laboratorio: `cd diveintoansible-lab && sudo docker compose up -d`

Importante

Esto creará contenedores interconectados. El nodo `ubuntu-c` actuará como tu controlador de Ansible.

Verificando el Entorno

El laboratorio de *diveintoansible* ya viene con **Ansible instalado** y las **llaves SSH preconfiguradas** entre el nodo de control y los demás nodos.

Ingresar al nodo de control y verificar:

Comandos

```
sudo docker exec -it ubuntu-c bash
ansible --version
ssh ubuntu1
```

¡Estamos listos para empezar a usar Ansible inmediatamente sin configuraciones manuales!

Bloque 4: Comandos Ad-Hoc

¿Qué son los Comandos Ad-Hoc?

Definición

Comandos **únicos** que se ejecutan directamente desde la terminal sin necesidad de escribir un playbook.

¿Cuándo usarlos?

- Tareas rápidas y puntuales.
- Verificar estado de servidores.
- Ejecutar un comando en múltiples máquinas al instante.



*“Una línea,
múltiples servidores.”*

El Archivo de Inventario

El inventario define **qué servidores** gestiona Ansible.

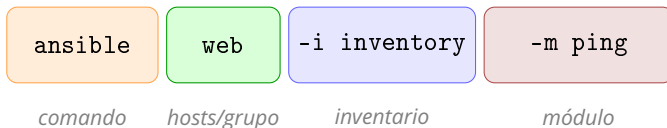
Inventario (inventory.ini)

```
[web]
ubuntu1
ubuntu2

[db]
centos1
centos2

[all:vars]
ansible_user=ansible
```

Sintaxis de Comandos Ad-Hoc



Estructura general

```
ansible <hosts> -i <inventario> -m <módulo> [-a «argumentos>"]
```

Ejemplos Prácticos de Comandos Ad-Hoc (1/3)

1. Verificar conectividad (módulo ping)

```
ansible all -i inventory.ini -m ping
```

Nota: No es un ping ICMP. Verifica que Ansible pueda abrir una sesión SSH exitosamente.

2. Ejecutar un comando remoto (módulo command)

```
ansible web -i inventory.ini -m command -a "uptime"
```

Ejemplos Prácticos de Comandos Ad-Hoc (2/3)

3. Instalar un paquete (módulo apt)

```
ansible web -i inventory.ini -m apt \
  -a "name=htop state=present" --become
```

4. Copiar un archivo a los servidores

```
ansible all -i inventory.ini -m copy \
  -a "src=./hola.txt dest=/tmp/hola.txt"
```

Ejemplos Prácticos de Comandos Ad-Hoc (3/3)

5. Recopilar información del sistema (facts)

```
ansible web -i inventory.ini -m setup
```

6. Reiniciar un servicio

```
ansible web -i inventory.ini -m service \
  -a "name=nginx state=restarted" --become
```

Módulos Más Usados en Ansible

Módulo	Descripción	Ejemplo
ping	Verificar conectividad	-m ping
command	Ejecutar comando	-a 'whoami'
shell	Ejecutar con shell	-a 'cat /etc/os-release'
copy	Copiar archivos	-a 'src=... dest=...'
apt	Gestión de paquetes	-a 'name=nginx state=present'
service	Gestión de servicios	-a 'name=nginx state=started'
setup	Recopilar facts	-m setup

Ansible tiene **miles de módulos** para casi cualquier tarea de administración.

Bloque 5: Primer Playbook

De Comandos Ad-Hoc a Playbooks

Comandos ad-hoc

- Ideales para pruebas y tareas rápidas.
- Se ejecutan una vez desde la terminal.

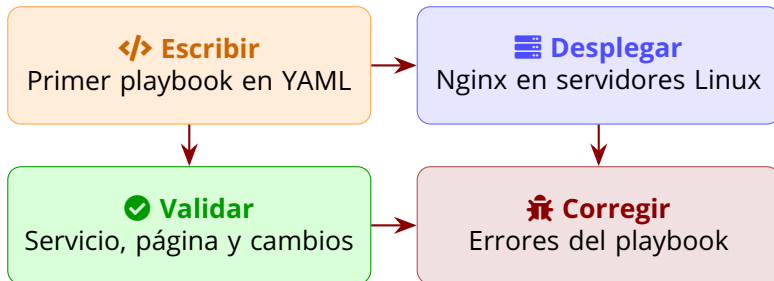
Playbooks

- Guardan la automatización en archivos YAML.
- Permiten repetir, revisar y versionar procedimientos.



**Automatización como
código**

Objetivo de la Sesión 2



¿Qué es un Playbook?

Definición

Un **playbook** es un archivo YAML que indica qué hosts serán gestionados y qué tareas ejecutará Ansible para llevarlos a un estado deseado.

Estructura mínima

```
---
- name: Mi primer playbook
  hosts: web
  become: true
  tasks:
    - name: Verificar conectividad
      ansible.builtin.ping:
```

Partes Principales de un Playbook

 **name**

Nombre legible del play

 **hosts**

Grupo o servidor objetivo

 **become**

Ejecutar con privilegios

 **tasks**

Lista de tareas

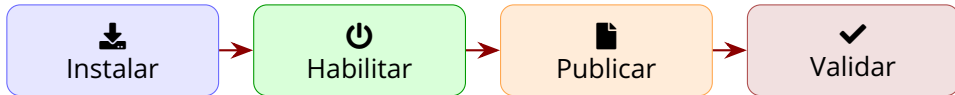
Anatomía de una Tarea

Una tarea = nombre + módulo + parámetros

```
- name: Instalar Nginx
  ansible.builtin.apt:
    name: nginx
    state: present
    update_cache: true
```

- `name`: describe qué hace la tarea.
- `ansible.builtin.apt`: módulo usado para gestionar paquetes.
- `state: present`: declara el estado deseado.

Flujo de Despliegue de Nginx



Meta

Dejar un servidor web funcionando con una página personalizada usando un solo comando de Ansible.

Playbook: Instalar y Habilitar Nginx

nginx.yml

```
---
- name: Desplegar Nginx
  hosts: web
  become: true
  tasks:
    - name: Instalar Nginx
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: true

    - name: Asegurar que Nginx esté activo
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
```


Publicar una Página HTML

Agregar una tarea con copy

```
- name: Publicar página principal
  ansible.builtin.copy:
    dest: /var/www/html/index.html
    content: |
      <h1>Servidor gestionado con Ansible</h1>
      <p>Despliegue automatizado de Nginx.</p>
    owner: www-data
    group: www-data
    mode: '0644'
```

Idempotencia: la Idea Clave

¿Qué significa?

Ejecutar el mismo playbook varias veces debe producir el **mismo estado final**, sin repetir cambios innecesarios.

- Si Nginx ya está instalado, Ansible no lo reinstala.
- Si el servicio ya está activo, no lo reinicia sin motivo.
- Si el archivo no cambió, no vuelve a copiarlo.



Repetible y seguro

Cómo Leer el Resultado de Ansible

Estado	Significado	Lectura rápida
ok	Ya estaba correcto	No hubo cambio
changed	Ansible modificó algo	Cambio aplicado
failed	La tarea falló	Revisar error
skipped	No se ejecutó	Condición no cumplida
unreachable	No conectó al host	Revisar SSH/inventario

La segunda ejecución de un buen playbook debería mostrar menos changed.

Ejecutar el Playbook

Comandos útiles

```
ansible-playbook -i inventory.ini nginx.yml
```

```
ansible-playbook -i inventory.ini nginx.yml --check
```

```
ansible-playbook -i inventory.ini nginx.yml -v
```

- Primera línea: aplica el playbook.
- `-check`: simula los cambios antes de ejecutarlos.
- `-v`: muestra más detalle para diagnosticar errores.

Validar el Despliegue

Desde el nodo de control

Verificamos que el servicio responda y que la página publicada esté disponible.

Validación rápida

```
ansible web -i inventory.ini -m command -a "systemctl status nginx"
ansible web -i inventory.ini -m uri -a "url=http://localhost"
```

- `systemctl status nginx`: confirma el estado del servicio.
- `uri`: prueba una respuesta HTTP desde el servidor.

Errores Básicos y Cómo Corregirlos

Error común	Causa probable	Qué revisar
UNREACHABLE	SSH o inventario	IP, usuario, llave
Error de YAML	Indentación incorrecta	Espacios y dos puntos
Permiso denegado	Falta privilegio	become: true
Paquete no encontrado	Caché desactualizada	update_cache
Servicio no inicia	Configuración inválida	Logs de Nginx

Mini Laboratorio de la Sesión 2

- 1 Crear el archivo `nginx.yml`.
- 2 Ejecutarlo contra el grupo `web`.
- 3 Ejecutarlo una segunda vez y comparar `ok` vs `changed`.
- 4 Cambiar el contenido del HTML y volver a ejecutar.
- 5 Validar el sitio con `curl` o el módulo `uri`.

Comando final

```
ansible-playbook -i inventory.ini nginx.yml
```

Resumen de la Sesión 2

- 1 Un **playbook** describe tareas en YAML para automatizar servidores de forma repetible.
- 2 Usamos `hosts`, `become` y `tasks` para estructurar la automatización.
- 3 Desplegamos **Nginx** instalando el paquete, habilitando el servicio y publicando HTML.
- 4 Entendimos la **idempotencia**: ejecutar varias veces sin repetir cambios innecesarios.
- 5 Practicamos ejecución, validación y corrección de errores básicos en playbooks.

Próxima Sesión

Sesión 3: Inventarios y Automatización Multi-Servidor — `hosts`, grupos, variables y MariaDB.

Objetivos de la Sesión 3

Organizar

Modelar hosts, grupos e información de conexión en un inventario legible.

Escalar

Ejecutar tareas sobre varios servidores sin duplicar playbooks.

Separar responsabilidades

Desplegar Nginx en el grupo web y MariaDB en el grupo de base de datos.

Reaccionar a cambios

Reiniciar servicios mediante handlers solo cuando una tarea notifique un cambio.

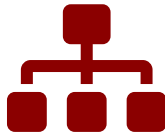
Inventarios estaticos

Del Host Aislado a la Infraestructura

Inventario

Fuente de verdad que indica a Ansible **que equipos gestiona, como se conecta y como se agrupan.**

- Un host puede pertenecer a varios grupos.
- Los grupos permiten dirigir tareas por funcion.
- Las variables eliminan datos repetidos de los comandos.



Inventario = alcance + contexto

Inventario INI: Hosts y Grupos

inventory.ini

```
[web]
web01 ansible_host=10.10.10.21
web02 ansible_host=10.10.10.22

[db]
db01 ansible_host=10.10.10.31

[linux:children]
web
db
```

linux es un grupo padre que incluye web y db.

Variables de Host y de Grupo

inventory.ini

```
[web]
web01 ansible_host=10.10.10.21 http_port=8081
web02 ansible_host=10.10.10.22 http_port=8082

[db]
db01 ansible_host=10.10.10.31 db_port=3306

[linux:vars]
ansible_user=ansible
ansible_python_interpreter=/usr/bin/python3
```

- Variables en la línea del host: particulares para ese servidor.
- [grupo:vars]: compartidas por los miembros del grupo.

Inspeccionar y Validar el Inventario

Ver estructura

```
ansible-inventory \
  -i inventory.ini \
  --graph
```

Ver datos completos

```
ansible-inventory \
  -i inventory.ini \
  --list
```

Probar conectividad

```
ansible linux \
  -i inventory.ini \
  -m ansible.builtin.ping
```

Si no aparece en `-graph`, el playbook no lo encontrara.

Patrones para Seleccionar Hosts

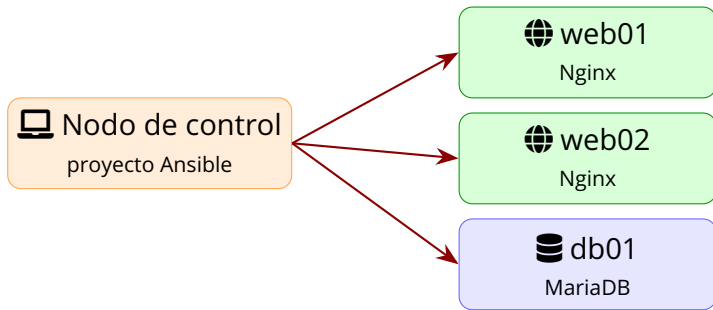
Patron	Alcance
all	Todos los hosts del inventario
web	Todos los miembros del grupo web
web01	Un host especifico
web:db	Union de ambos grupos
linux:!db	Linux excepto el grupo db
web[0]	Primer host del grupo web

Precaucion

Verifica el patron con `ansible <patron> -list-hosts` antes de una tarea destructiva.

Automatizacion multi-servidor

Un Proyecto, Varios Roles de Servidor



Principio

El inventario define el alcance; cada play aplica el estado correcto al grupo correcto.

Playbook con Varios Plays

site.yml

```
---
- name: Configurar servidores web
  hosts: web
  become: true
  tasks:
    - name: Instalar Nginx
      ansible.builtin.apt:
        name: nginx
        state: present

- name: Configurar servidor de base de datos
  hosts: db
  become: true
  tasks:
    - name: Instalar MariaDB
      ansible.builtin.apt:
        name: mariadb-server
        state: present
```

Ejecucion Controlada con limit y serial

Limitar desde la terminal

```
ansible-playbook \  
  -i inventory.ini \  
  site.yml \  
  --limit web01
```

Procesar por lotes

```
- name: Actualizar web  
  hosts: web  
  serial: 1  
  become: true
```

Uso

`-limit` reduce el alcance de una ejecucion; `serial` controla cuantos hosts se procesan a la vez.

Forks y Paralelismo

ansible.cfg

```
[defaults]
inventory = inventory.ini
forks = 10
host_key_checking = False
```

- Ansible ejecuta una tarea en varios hosts en paralelo.
- `forks` limita la cantidad de conexiones concurrentes.
- Aumentar paralelismo no corrige una red lenta ni un servicio fragil.

MariaDB en un servidor separado

Por que Separar Web y Base de Datos

Grupo web

Atiende solicitudes HTTP, publica contenido y puede escalar horizontalmente.

Grupo db

Conserva datos, requiere controles de acceso y cambios mas cuidadosos.

Separacion de responsabilidades

Un playbook multi-servidor no instala todo en todos los nodos: dirige cada configuracion al grupo adecuado.

Instalar y Habilitar MariaDB

Play para el grupo db

```
- name: Desplegar MariaDB
  hosts: db
  become: true
  tasks:
    - name: Instalar MariaDB y cliente Python
      ansible.builtin.apt:
        name:
          - mariadb-server
          - python3-pymysql
        state: present
        update_cache: true

    - name: Iniciar y habilitar MariaDB
      ansible.builtin.service:
        name: mariadb
        state: started
        enabled: true
```

Configurar MariaDB desde el Proyecto

Tarea de configuracion

```
- name: Permitir conexiones en la red del laboratorio
  ansible.builtin.lineinfile:
    path: /etc/mysql/mariadb.conf.d/50-server.cnf
    regexp: '^bind-address'
    line: 'bind-address = 0.0.0.0'
    backup: true
  notify: Reiniciar MariaDB
```

Produccion

Restringe red y privilegios; protege secretos con Ansible Vault.

Validar MariaDB

Estado del servicio

```
ansible db -b \  
-m ansible.builtin.command \  
-a "systemctl is-active mariadb"
```

Puerto en escucha

```
ansible db -b \  
-m ansible.builtin.shell \  
-a "ss -lntp | grep 3306"
```

Criterio

El playbook termina sin fallos, el servicio esta activo y el puerto esperado aparece en el nodo correcto.

Handlers y cambios reales

Que Problema Resuelve un Handler

Sin handler

Reiniciar el servicio en cada ejecucion, aunque la configuracion no haya cambiado.

Con handler

Una tarea notifica el reinicio solo cuando reporta changed.

- Evita interrupciones innecesarias.
- Agrupa varias notificaciones en una sola ejecucion.
- Se ejecuta normalmente al final del play.



Cambio → notificacion

notify y handlers

Configuracion y reaccion

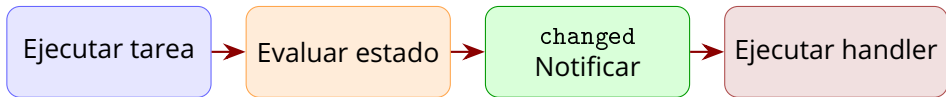
tasks:

- name: Configurar bind-address
 - ansible.builtin.lineinfile:
 - path: /etc/mysql/mariadb.conf.d/50-server.cnf
 - regexp: '^bind-address'
 - line: 'bind-address = 0.0.0.0'
 - notify: Reiniciar MariaDB

handlers:

- name: Reiniciar MariaDB
 - ansible.builtin.service:
 - name: mariadb
 - state: restarted

Ciclo de un Handler



Idempotencia observable

En la segunda ejecución, si el archivo ya contiene el valor correcto, la tarea queda en `ok` y el handler no se ejecuta.

Forzar Handlers Antes del Final

meta: flush_handlers

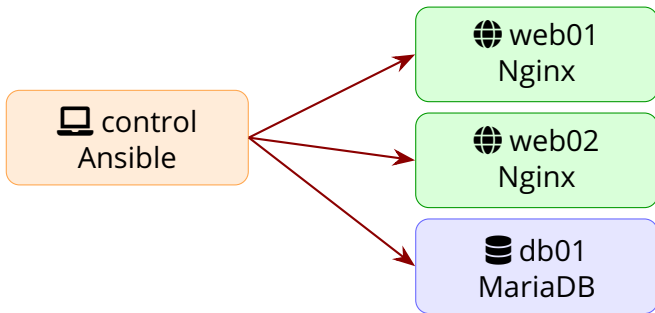
```
- name: Aplicar configuracion de MariaDB
  ansible.builtin.template:
    src: 50-server.cnf.j2
    dest: /etc/mysql/mariadb.conf.d/50-server.cnf
  notify: Reiniciar MariaDB

- name: Ejecutar handlers pendientes ahora
  ansible.builtin.meta: flush_handlers

- name: Validar el puerto despues del reinicio
  ansible.builtin.wait_for:
    port: 3306
    timeout: 30
```

Laboratorio integrador

Arquitectura del Laboratorio



Resultado esperado

Dos nodos web configurados en paralelo y un nodo de base de datos con reinicio condicionado por cambios.

Etapa 1: Preparar y Comprobar

Preparar

```
cd "en vivo/laboratorio-sesion-03"  
ansible-inventory -i inventory.ini --graph  
ansible all -i inventory.ini --list-hosts
```

Comprobar

```
ansible linux -i inventory.ini -m ansible.builtin.ping  
ansible all -i inventory.ini -m ansible.builtin.command -a "hostname"
```

Etapa 2: Ejecutar el Despliegue

Validar

```
ansible-playbook -i inventory.ini site.yml --syntax-check
ansible-playbook -i inventory.ini site.yml --check
```

Aplicar

```
ansible-playbook -i inventory.ini site.yml
ansible-playbook -i inventory.ini site.yml
```

La segunda ejecucion debe reducir `changed` y no reiniciar MariaDB.

Etapa 3: Validar y Corregir

Servicios

```
ansible web -b \  
  -m command \  
  -a "systemctl is-active nginx"  
  
ansible db -b \  
  -m command \  
  -a "systemctl is-active mariadb"
```

Diagnostico

```
ansible-playbook \  
  -i inventory.ini \  
  site.yml -v  
  
ansible web \  
  -i inventory.ini \  
  --list-hosts
```

Errores Frecuentes

Sintoma	Causa probable	Revision
No se seleccionan hosts UNREACHABLE	Grupo mal escrito IP, usuario o llave SSH	-graph y -list-hosts Variables de conexion
MariaDB en nodos web	hosts incorrecto	Separar plays por grupo
Handler nunca corre	La tarea no cambia o el nombre no coincide	notify y handlers
Handler corre siempre	Tarea no idempotente	Usar modulo declarativo

Resumen de la Sesion

- 1 El inventario organiza hosts, grupos, grupos padre y variables de conexion.
- 2 Los patrones, `-limit`, `serial` y `forks` controlan el alcance y paralelismo.
- 3 Un playbook multi-servidor aplica Nginx al grupo web y MariaDB al grupo db.
- 4 Los handlers reaccionan a `changed` y evitan reinicios innecesarios.
- 5 La segunda ejecucion permite comprobar la idempotencia del proyecto.

Proxima sesion

Sesion 4: variables, facts, condicionales, loops y templates Jinja2.

¿Qué son las Variables en Ansible?

Definición

Las variables permiten escribir playbooks **genéricos y reutilizables**, adaptando su comportamiento sin cambiar la lógica del código.

¿Para qué sirven?

- Cambiar puertos, rutas o usuarios por entorno.
- Evitar repetir valores en el playbook.



*"Un playbook,
múltiples entornos."*

Tipos y Ubicaciones de Variables



En el Playbook

Directamente en la sección `vars:` del play.



`group_vars/`

Variables compartidas por un grupo de hosts.



`host_vars/`

Variables exclusivas para un host específico.

`>_ Línea de Comandos`

Pasadas con la opción `-e` al ejecutar.

Variables Definidas en el Playbook

playbook.yml

```
- name: Configurar servidor web
  hosts: webservers
  become: yes
  vars:
    puerto_http: 80
    nombre_paquete: nginx
    directorio_web: /var/www/html
  tasks:
    - name: Instalar {{ nombre_paquete }}
      apt:
        name: "{{ nombre_paquete }}"
        state: present
```

Sintaxis

Las variables se referencian con dobles llaves:

```
{{ nombre_var }}
```

Ventaja

Cambiar `nombre_paquete` en un solo lugar actualiza todas las tareas que lo usan.

Variables por Grupo y por Host

group_vars/webservers.yml

Variables para **todos** los hosts del grupo:

```
puerto_http: 80
nombre_app: "MiApp"
entorno: produccion
```

Archivo: group_vars/webservers.yml
Se aplica a todos en el grupo [webservers]

host_vars/ubuntu1.yml

Variables **exclusivas** para un host:

```
entorno: "produccion"
memoria_max: "512m"
ip_publica: "10.0.0.5"
```

Archivo: host_vars/ubuntu1.yml
Solo se aplica al host ubuntu1

¿Qué son los Facts?

Definición

Los **Facts** son variables que Ansible **descubre automáticamente** al conectarse a un host, antes de ejecutar cualquier tarea.

Información que recoge Ansible:

- Sistema operativo y versión.
- Dirección IP de cada interfaz de red.
- Cantidad de CPUs y RAM disponible.
- Hostname, arquitectura, tiempo activo.



**Ansible descubre
el sistema
por ti**

Explorando los Facts del Sistema

Ver todos los facts de un host:

Comando Ad-Hoc

```
ansible ubuntu1 -i inventory.ini -m setup
```

Facts más útiles en la práctica:

Fact	Contenido
ansible_hostname	Nombre del host
ansible_os_family	Familia del SO (Debian, RedHat...)
ansible_distribution	Distribución exacta (Ubuntu, CentOS...)
ansible_default_ipv4.address	IP principal del host

Usando Facts en Playbooks

Mostrar IP y SO de cada servidor

```
- name: Mostrar información del sistema
  hosts: all
  tasks:
    - name: Info del host
      debug:
        msg: >
          Host: {{ ansible_hostname }},
          IP: {{ ansible_default_ipv4.address }},
          SO: {{ ansible_distribution }} {{ ansible_distribution_version }}
```

Los facts se usan igual que cualquier variable: con dobles llaves {{ }}.

Condicionales con `when`

¿Para qué sirven?

Ejecutar una tarea **solo si se cumple** una condición. Combinados con Facts, permiten que el mismo playbook funcione en Ubuntu y CentOS.

Ejemplo

```
- name: Instalar en Debian/Ubuntu
  apt:
    name: nginx
    state: present
  when: ansible_os_family == "Debian"
```



*Un playbook
para todos
los servidores*

Repetición de Tareas con `loop`

¿Cuándo usar `loop`?

Ejecutar la **misma acción** sobre una lista de elementos (paquetes, usuarios, directorios) sin repetir código.

Sin `loop` (repetitivo)

```
- apt:
  name: curl
  state: present
- apt:
  name: git
  state: present
- apt:
  name: vim
  state: present
```

Con `loop` (elegante)

```
- name: Instalar paquetes
  apt:
    name: "{{ item }}"
    state: present
  loop:
    - curl
    - git
    - vim
```

Combinando `when` y `loop` (1/2)

Instalar paquetes solo en Ubuntu

```
- name: Instalar herramientas en Ubuntu
  apt:
    name: "{{ item }}"
    state: present
  loop:
    - htop
    - net-tools
    - iotop
  when: ansible_distribution == "Ubuntu"
```

Ansible ejecuta la tarea **3 veces** (una por ítem) pero **solo si** el host es Ubuntu.

Combinando `when` y `loop` (2/2)

Crear directorios de la aplicación

```
- name: Crear directorios del proyecto
  file:
    path: "{{ item }}"
    state: directory
    mode: '0755'
  loop:
    - /var/www/html
    - /var/log/miapp
    - /etc/miapp/config
```

El módulo `file` crea los 3 directorios en **una sola tarea**.

¿Qué son los Templates Jinja2?

Definición

Los **Templates** son archivos de texto con *marcadores* que Ansible reemplaza con variables reales antes de enviarlos al servidor.

Ventajas:

- Un solo archivo para múltiples entornos.
- Variables, condicionales y loops **dentro** del archivo.
- Motor **Jinja2** (mismo que Flask, Django).



nginx.conf.j2



nginx.conf
(con valores reales)

Extensión .j2 — se guardan en templates/

Sintaxis Jinja2 Esencial

Variables

```
{{ nombre_var }}
```

Inserta el valor de la variable

Condicionales

```
{% if condicion %}
```

Bloque condicional

Bucles

```
{% for x in lista %}
```

Itera sobre una lista

Comentarios

```
{# comentario #}
```

No se renderiza

Ejemplo: Template de Nginx

Archivo: templates/nginx.conf.j2

```
server {
    listen {{ puerto_http }};
    server_name {{ ansible_hostname }};

    root {{ directorio_web }};
    index index.html;

    {% if entorno == 'produccion' %}
    access_log /var/log/nginx/access_prod.log;
    error_log /var/log/nginx/error_prod.log;
    {% else %}
    access_log /var/log/nginx/access_dev.log;
    {% endif %}
}
```

Aplicando el Template con la Tarea

Tarea en el playbook

```
- name: Generar configuración de Nginx
  template:
    src: templates/nginx.conf.j2
    dest: /etc/nginx/sites-available/default
    owner: root
    group: root
    mode: '0644'
  notify: Reiniciar Nginx
```

Flujo Completo: Variables + Facts + Templates

Variables

Definidas en `vars:`,
`group_vars/` o `host_vars/`

Facts

Descubiertos automáticamente: OS, IP, RAM...

when / loop

Lógica condicional y
repetición de tareas

Template .j2

Configuraciones dinámicas
por host y entorno

Resultado

Un playbook inteligente que adapta su comportamiento **automáticamente** según el servidor, el entorno y los datos del sistema.

Resumen de la Sesión 4

- 1 Variables:** definidas en el playbook, `group_vars/` o `host_vars/`, referenciadas con `{{ }}`.
- 2 Facts:** descubiertos automáticamente por Ansible al inicio de cada play (OS, IP, RAM...).
- 3 Condicionales** (`when`) y **Loops** (`loop`): lógica y repetición sin duplicar código.
- 4 Templates Jinja2:** archivos `.j2` que generan configuraciones dinámicas personalizadas por host y entorno.

Próxima Sesión

Sesión 5: Roles, Monitoreo y Buenas Prácticas — organización profesional del proyecto con roles, Ansible Galaxy y automatización de Prometheus.

Roles, Monitoreo y Buenas Prácticas

Objetivos de la Sesión 5

Organizar y reutilizar

- Comprender la estructura profesional de un rol.
- Separar `tasks`, `handlers`, `templates`, `defaults` y `vars`.
- Refactorizar playbooks monolíticos en componentes mantenibles.

Compartir y observar

- Crear e instalar roles con Ansible Galaxy.
- Automatizar Node Exporter en los servidores administrados.
- Integrar métricas de infraestructura con Prometheus.

¿Qué son los Roles en Ansible?

- Un playbook grande mezcla instalación, configuración y reinicios en un único archivo.
- Un **rol** empaqueta una responsabilidad concreta en una estructura estándar y reutilizable.
- Ejemplos: `common`, `webserver`, `database`, `node_exporter`.
- El mismo rol puede aplicarse a distintos inventarios y entornos sin duplicar tareas.

Idea central: responsabilidad única y variables configurables.



Modularidad y orden

Estructura Profesional de un Rol

tasks/main.yml

Flujo principal
de tareas del rol.

handlers/main.yml

Acciones ejecutadas
al recibir notify.

defaults/ y vars/

Valores configurables
y variables internas.

templates/ y files/

Jinja2 y archivos estáticos
que distribuye el rol.

También pueden incluirse `meta/`, `tests/` y `README.md`.

Handlers dentro de un Rol

Tarea que notifica el cambio

```
- name: Publicar nginx.conf
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
    mode: '0644'
  notify: Reiniciar nginx
```

handlers/main.yml

```
- name: Reiniciar nginx
  ansible.builtin.service:
    name: nginx
    state: restarted
```

Comportamiento idempotente

El handler se ejecuta al final del play y únicamente cuando la plantilla cambia.

Buenas Prácticas: defaults frente a vars

defaults/main.yml

- Menor precedencia del rol.
- Valores pensados para ser sobrescritos.
- Puertos, versiones, rutas y opciones públicas.

```
node_exporter_port: 9100
```

vars/main.yml

- Mayor precedencia que defaults.
- Uso reservado para detalles internos.
- Evitar colocar aquí valores que el usuario necesite cambiar.

```
node_exporter_service:  
node_exporter
```

Creación y Uso de Roles con Ansible Galaxy

Comandos principales

```
# Crear la estructura local de un rol
ansible-galaxy role init roles/node_exporter
# Buscar e inspeccionar contenido comunitario
ansible-galaxy search node_exporter
ansible-galaxy role info cloudalchemy.node_exporter
# Instalar las dependencias declaradas por el proyecto
ansible-galaxy role install -r requirements.yml -p roles/
```

Antes de usar un rol comunitario

Revisar mantenedor, documentación, plataformas soportadas, variables, licencia y actividad reciente.

Dependencias Reproducibles con `requirements.yml`

roles:

- name: node_exporter
src: cloudalchemy.node_exporter
version: "0.23.3"

collections:

- name: community.general
version: "10.4.0"

¿Por qué fijar versiones?

- Evita cambios inesperados.
- Permite repetir el despliegue en CI/CD.
- Facilita auditorías y actualizaciones controladas.

Refactorización del Proyecto a Roles

Playbook principal: `site.yml`

```
- name: Configuración comun
  hosts: all
  become: true
  roles:
    - common
    - node_exporter

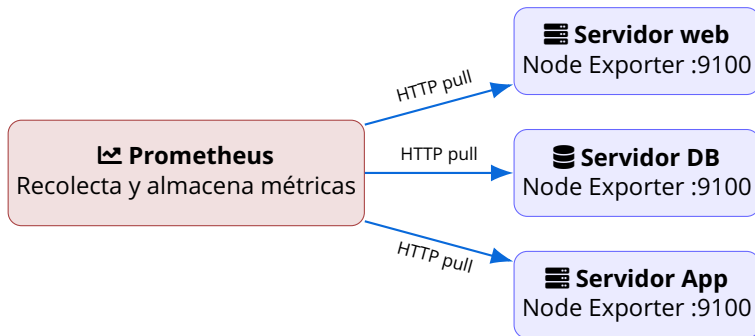
- name: Capa web
  hosts: web
  become: true
  roles:
    - webserver

- name: Base de datos
  hosts: db
  become: true
  roles:
    - database
```

Resultado

- El inventario define el alcance.
- Cada rol concentra una responsabilidad.
- Las variables ajustan cada entorno.
- Los handlers reaccionan solo a cambios reales.
- `site.yml` expresa la arquitectura con claridad.

Arquitectura de Monitoreo Automatizado



Modelo pull

Prometheus consulta periódicamente `/metrics`; Node Exporter no necesita enviar datos al servidor central.

Rol node_exporter: Instalación y Servicio

tasks/main.yml

- name: Crear usuario del servicio
 ansible.builtin.user:
 name: node_exporter
 system: true
 shell: /usr/sbin/nologin
- name: Instalar binario
 ansible.builtin.copy:
 src: node_exporter
 dest: /usr/local/bin/node_exporter
 owner: root
 group: root
 mode: '0755'
- name: Publicar unidad systemd
 ansible.builtin.template:
 src: node_exporter.service.j2
 dest: /etc/systemd/system/node_exporter.service
 notify: Reiniciar node exporter

templates/node_exporter.service.j2

```
[Unit]
Description=Prometheus Node Exporter
After=network-online.target

[Service]
User=node_exporter
ExecStart=/usr/local/bin/node_exporter \
  --web.listen-address={{ node_exporter_port }}
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Estado deseado

- ansible.builtin.systemd_service:
 name: node_exporter
 enabled: true
 state: started
 daemon_reload: true

Configurar Prometheus con los Hosts Administrados

Fragmento de `prometheus.yml.j2`

```
scrape_configs:
  - job_name: "linux_nodes"
    static_configs:
      - targets:
{% for host in groups['linux'] %}
        - "{{ hostvars[host].ansible_host }}:9100"
{% endfor %}
```

Ventaja del template

- El inventario es la fuente de verdad.
- Los nuevos hosts se agregan automáticamente.
- El cambio notifica un handler para recargar Prometheus.

Validación e Idempotencia del Monitoreo

Comprobaciones

```
ansible linux -b -m command \
  -a "systemctl is-active node_exporter"
```

```
ansible linux -m uri -a \
  "url=http://localhost:9100/metrics"
```

```
ansible-playbook -i inventory.ini \
  site.yml --check --diff
```

Criterios de éxito

- Servicio active y enabled.
- Puerto 9100 accesible solo desde la red autorizada.
- Target en estado UP en Prometheus.
- Segunda ejecución con `changed=0`.

Errores Frecuentes y Buenas Prácticas

Problema	Causa habitual	Práctica recomendada
Rol monolítico	Demasiadas responsabilidades	Separar por servicio o capacidad
Variables rígidas	Valores públicos dentro de vars/	Exponerlos en defaults/
Reinicios en cada corrida	Uso de tareas directas	Aplicar <code>notify</code> y <code>handlers</code>
Galaxy impredecible	Versiones no fijadas	Mantener <code>requirements.yml</code>
Target Prometheus DOWN	Servicio, firewall o IP incorrectos	Validar <code>systemd</code> , puerto y endpoint
Secretos en el rol	Credenciales en texto plano	Usar Ansible Vault

Resumen de la Sesión 5

- Organizamos proyectos mediante roles con `tasks`, `handlers`, `templates`, `defaults` y `vars`.
- Refactorizamos el playbook para hacerlo ordenado, reutilizable y escalable.
- Utilizamos Ansible Galaxy y `requirements.yml` para gestionar dependencias reproducibles.
- Automatizamos Node Exporter y la configuración de targets de Prometheus.
- Validamos servicio, métricas, conectividad e idempotencia del despliegue.

Próxima sesión

Sesión 6: Seguridad, Backups y Despliegue Final — Vault, hardening SSH, UFW, Fail2Ban, copias programadas y cierre del proyecto.

Seguridad, Backups y Despliegue Final

Objetivos de la Sesión 6

Proteger la infraestructura

- Cifrar secretos con Ansible Vault.
- Aplicar hardening SSH sin perder acceso.
- Restringir puertos con UFW.
- Bloquear intentos abusivos con Fail2Ban.

Cerrar el proyecto

- Programar y validar backups de MariaDB.
- Orquestrar todos los roles desde `site.yml`.
- Ejecutar pruebas de aceptación.
- Demostrar idempotencia en el laboratorio final.

Cifrado de Secretos con Ansible Vault

- Contraseñas, tokens y claves no deben almacenarse en texto plano.
- **Ansible Vault** cifra archivos o variables sensibles mediante AES-256.
- El contenido se descifra en memoria durante la ejecución del playbook.
- El repositorio conserva código versionable sin revelar credenciales.



Secretos cifrados

Regla de seguridad

El archivo de contraseña del Vault nunca se versiona ni se distribuye junto al repositorio.

Flujo de Trabajo con Ansible Vault

Administrar secretos

```
# Crear el archivo cifrado
ansible-vault create group_vars/all/vault.yml

# Consultar o editar
ansible-vault view group_vars/all/vault.yml
ansible-vault edit group_vars/all/vault.yml

# Cambiar la clave
ansible-vault rekey group_vars/all/vault.yml
```

Ejecutar el proyecto

```
# Solicitar clave interactivamente
ansible-playbook site.yml \
    --ask-vault-pass

# Automatización controlada
ansible-playbook site.yml \
    --vault-password-file .vault-pass
```

.gitignore

Añadir `.vault-pass`, llaves privadas, reintentos y archivos temporales.

Hardening SSH: Orden Seguro de Aplicación

- 1 Crear o confirmar el usuario administrativo remoto.
- 2 Instalar y probar la llave pública con `authorized_key`.
- 3 Validar una segunda conexión SSH en otra terminal.
- 4 Desactivar `PermitRootLogin` y `PasswordAuthentication`.
- 5 Validar `sshd -t` antes de recargar el servicio.
- 6 Aplicar UFW permitiendo SSH antes de habilitar la política `deny`.

Evitar el `lockout`

Nunca cierres la sesión actual hasta confirmar que la autenticación por llave funciona y el puerto SSH permanece permitido.

Rol de Seguridad: SSH y UFW

Configurar SSH de forma validada

- name: Publicar configuracion SSH
ansible.builtin.template:
 src: sshd_config.j2
 dest: /etc/ssh/sshd_config
 owner: root
 group: root
 mode: '0600'
 validate: '/usr/sbin/sshd -t -f %s'
notify: Recargar ssh
- name: Instalar llave autorizada
ansible.posix.authorized_key:
 user: "{{ admin_user }}"
 key: "{{ lookup('file', admin_pubkey) }}"

Permitir solo servicios necesarios

- name: Permitir puertos autorizados
community.general.ufw:
 rule: allow
 port: "{{ item }}"
 proto: tcp
 loop: "{{ firewall_allowed_ports }}"
- name: Activar UFW con politica deny
community.general.ufw:
 state: enabled
 policy: deny
 direction: incoming

Valor sugerido

firewall_allowed_ports: ['22', '80', '443']

Protección frente a Intrusos con Fail2Ban

- Analiza logs de autenticación.
- Detecta intentos fallidos repetidos.
- Aplica bloqueos temporales mediante el firewall.
- Complementa, pero no reemplaza, la autenticación por llaves.

```
- name: Instalar Fail2Ban
  ansible.builtin.package:
    name: fail2ban
    state: present

- name: Configurar jail SSH
  ansible.builtin.template:
    src: jail.local.j2
    dest: /etc/fail2ban/jail.local
  notify: Reiniciar fail2ban
```

Valores iniciales

`maxretry=3`, `findtime=10m` y `bantime=1h`; deben ajustarse al riesgo y operación del entorno.

Backups Automatizados y Verificables

- name: Crear directorio de backups

ansible.builtin.file:

path: /var/backups/mariadb

state: directory

owner: root

group: root

mode: '0700'

- name: Instalar script de backup

ansible.builtin.template:

src: backup_mariadb.sh.j2

dest: /usr/local/sbin/backup_mariadb

mode: '0700'

- name: Programar backup diario

ansible.builtin.cron:

name: backup_mariadb

minute: '0'

hour: '2'

user: root

job: /usr/local/sbin/backup_mariadb

- name: Aplicar retencion de 7 dias

ansible.builtin.cron:

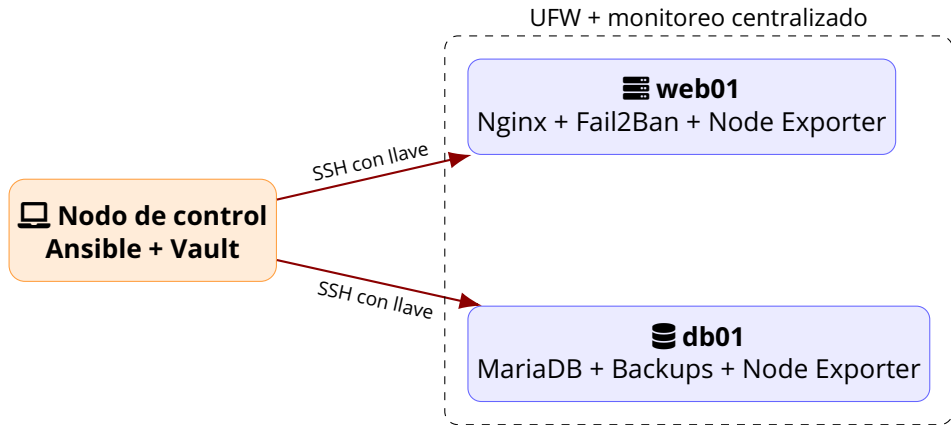
name: limpiar_backups

minute: '30'

hour: '2'

job: "find /var/backups/mariadb \
-type f -mtime +7 -delete"

Arquitectura del Proyecto Final



Laboratorio Final: Alcance y Entregables

Objetivo

Desplegar una infraestructura segura y observable mediante un único comando, reutilizando los roles construidos durante el curso.

- Web: Nginx y contenido dinámico.
- DB: MariaDB y backup programado.
- Todos: hardening, UFW y Node Exporter.

Entregables

- Inventario y configuración de Ansible.
- Roles documentados y parametrizados.
- Secretos cifrados con Vault.
- `site.yml` y `requirements.yml`.
- Evidencias de validación e idempotencia.

Laboratorio Final: Estructura del Repositorio

```
ansible-final/  
|-- ansible.cfg  
|-- inventory.ini  
|-- requirements.yml  
|-- site.yml  
|-- group_vars/  
|   |-- all/main.yml  
|   '-- all/vault.yml          # cifrado  
'-- roles/  
    |-- common/  
    |-- security/  
    |-- webserver/  
    |-- database/  
    |-- node_exporter/  
    '-- backup/
```

Control de versiones

Versionar código, templates y archivos Vault cifrados. Excluir claves privadas, `.vault-pass`, reintentos y artefactos temporales.

Laboratorio Final: Inventario y Variables

inventory.ini

```
[web]
web01 ansible_host=10.10.10.21
```

```
[db]
db01 ansible_host=10.10.10.31
```

```
[linux:children]
web
db
```

```
[linux:vars]
ansible_user=ansible
ansible_python_interpreter=/usr/bin/python3
```

group_vars/all/main.yml

```
admin_user: ansible
firewall_allowed_ports: ['22']
node_exporter_port: 9100
backup_retention_days: 7
vault_db_password: "{{ vault_mariadb_password }}"
```

group_vars/all/vault.yml

```
vault_mariadb_password: "CAMBIAR_Y_CIFRAR"
```

```
ansible-vault encrypt group_vars/all/vault.yml
```

Laboratorio Final: Orquestación con `site.yml`

```
- name: Base comun y seguridad
  hosts: linux
  become: true
  roles:
    - common
    - security
    - node_exporter
```

```
- name: Capa web
  hosts: web
  become: true
  roles:
    - webserver
```

```
- name: Capa de datos
  hosts: db
  become: true
  roles:
    - database
    - backup
```

Orden de despliegue

Primero acceso y seguridad comunes; después servicios por grupo. Los handlers se ejecutan únicamente ante cambios.

Laboratorio Final: Pipeline de Ejecución

- 1 Instalar dependencias y validar inventario.

```
ansible-galaxy install -r requirements.yml
ansible-inventory -i inventory.ini --graph
ansible linux -i inventory.ini -m ping
```

- 2 Validar sintaxis y simular cambios.

```
ansible-playbook -i inventory.ini site.yml --syntax-check
ansible-playbook -i inventory.ini site.yml --check --diff --ask-vault-pass
```

- 3 Desplegar y guardar evidencia.

```
ansible-playbook -i inventory.ini site.yml --ask-vault-pass | tee despliegue.log
```

Laboratorio Final: Matriz de Validación

Componente	Prueba	Resultado esperado
SSH	Nueva conexión con llave	Acceso exitoso; root y password deshabilitados
UFW	<code>ufw status</code>	Solo puertos autorizados
Nginx	Petición HTTP/HTTPS	Código 200 y contenido correcto
MariaDB	Estado y consulta simple	Servicio activo; autenticación con Vault
Fail2Ban	<code>fail2ban-client status sshd</code>	Jail habilitada y operativa
Node Exporter	<code>/metrics</code> en puerto 9100	Métricas disponibles desde Prometheus
Backup	Ejecutar script y listar destino	Archivo no vacío con permisos restringidos

Laboratorio Final: Cambio Controlado e Idempotencia

Primera ejecución

Registrar `ok`, `changed`, `failed` y `unreachable`. Corregir cualquier fallo antes de continuar.

Segunda ejecución

```
ansible-playbook -i inventory.ini \
  site.yml --ask-vault-pass
```

Resultado esperado: `changed=0`.

Cambio controlado

Modificar el contenido web o un parámetro permitido. Debe cambiar únicamente el host y rol afectados; el handler correspondiente se ejecuta una sola vez.

Evidencia

Comparar los tres recap: despliegue inicial, segunda corrida idempotente y cambio controlado.

Errores Frecuentes del Laboratorio Final

Síntoma	Causa probable	Revisión
UNREACHABLE	IP, usuario o llave incorrectos	Inventario, SSH y permisos de la llave
Lockout SSH	UFW o hardening aplicado antes de validar	Consola, llave autorizada y puerto 22
Vault no descifra Handler no corre	Clave o archivo equivocado Tarea sin cambio o nombre distinto	Vault ID y origen de la contraseña notify, listen y resultado
Target DOWN	Servicio o firewall	systemd, puerto 9100 y Prometheus
Backup vacío Segunda corrida cambia	Credenciales, ruta o permisos Tarea no declarativa	Script manual, logs y restauración Módulo, condiciones y templates

Cierre de la Sesión 6 y del Curso

- Protegimos secretos mediante Ansible Vault.
- Aplicamos hardening SSH, UFW y Fail2Ban sin perder acceso administrativo.
- Automatizamos backups de MariaDB con retención y validación.
- Integramos todos los roles en un despliegue final reproducible.
- Comprobamos funcionalidad, seguridad, monitoreo e idempotencia.

Proyecto completado

La infraestructura ya puede reconstruirse desde código, auditarse mediante Git y evolucionar de forma controlada hacia CI/CD y producción.